

第2章 运算符与表达式

建议学时:3-4 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立 Rust 运算符全景:算术、位、比较、逻辑、赋值、范围——知道哪些与 C 相同、哪些被移除、哪些是新增
- 掌握 **溢出行为的明确规则**: debug 模式 panic, release 模式回绕;以及 wrapping/checked/saturating 三种显式选择
- 理解整数除法与取模的截断语义(与 C 一致,与 Python 不同);负数的 % 到底等于几
- 掌握 **"Rust 没有 ++ 与 --"**,以及自增自减在 C 里造成的两类经典 bug 为何在 Rust 消失
- 掌握位运算全套,并知道 Rust 优先级规则修复了 C 的 `x & 1 == 0` 经典坑
- 掌握 if/loop/match 都是表达式: **C 的三目 ?: 在 Rust 中就是 if 表达式**;赋值是语句不是表达式
- 认识 Rust 特色运算符: ? 错误传播、范围 .. 与 ..=、复合赋值;了解 Rust 规定的求值顺序

💡 从 C 到 Rust:本章主题如何迁移

运算符是 C 程序员最熟悉的领域——但恰恰因为熟悉,C 的坏习惯最容易在这里原样搬进来。Rust 对运算符做的第一件事是**清理 C 的未定义行为**:有符号整数溢出从"UB,一切皆有可能"变成"debug 模式 panic、release 模式回绕"的确定性规则;子表达式的求值顺序从"未指定"变成"明确规定,从左到右"。你在 C 里依赖编译器脾气写的表达式,在 Rust 里要么是编译错误,要么行为确定。

第二件事是**删除和新增**。Rust 没有 ++/--(它们是 C 里 `i = i++ + i++` 这类灾难的温床),没有三目 ?: (if 表达式顶替),但有范围运算符 `1..5`、错误传播运算符 `?`。每个新增运算符都是"消灭一类样板代码"——`?` 消灭的正是 C 里 `if (err) return err;` 的复制粘贴洪流。

第三件事是**表达式化**。C 里 if/while 是语句,只能"做事";Rust 里 if、match、loop、代码块 `{}` 都是表达式,能"算出值"。于是 `int max = a > b ? a : b;` 变成 `let max = if a > b { a } else { b };` ——同样的语义,更少的括号,且**没有赋值语句混进条件里**的可能。

最后提醒一句贯穿全书的风格差异:C 允许 `if (n)`、`if (ptr)` 依赖"非零即真";Rust 的条件必须是 bool, `&& / ||` 永远返回 bool(不像 Python/JS 返回操作数)。刚开始会嫌啰嗦,但一周后你会感谢编译器替你挡住"把指针当布尔用"这类错误。

📦 前置知识自查

- C 的算术/位/逻辑/比较/赋值运算符与优先级表(能默写更好)
- C 的自增自减:前置/后置语义(`i++` 与 `++i`),以及在表达式中混用的危害
- C 有符号溢出是 UB、整数除法向零截断、负数取模的行为
- C 的短路求值 `&& / ||`,以及 `(x & 1) == 0` 为什么要加括号
- C 的三目运算符 `?:` 与逗号运算符

🚀 本章学习路线

1. **第一阶段(约1小时):**读 §2.1 运算符总览与优先级表,对照 C 划出"删除了什么、新增了什么"
2. **第二阶段(约1.5小时):**重点攻 §2.2 溢出与 §2.4 比较逻辑——这两节是 C 程序员最容易踩坑的地方
3. **第三阶段(约1小时):**§2.5–§2.7 表达式化与特色运算符,亲手改写几个 C 表达式
4. **验收标准:**能把 10 个 C 表达式逐一翻译成 Rust,并说出每处翻译的理由

本章知识导图

- §2.1 运算符总览:与 C 的差异清单、完整优先级表
- §2.2 算术运算:溢出规则(debug panic / release 回绕)、除法与取模截断
- §2.3 位运算:& | ^ ! << >>、移位语义、标志位生产场景
- §2.4 比较与逻辑:值比较、浮点比较、短路、无隐式真值
- §2.5 赋值:复合赋值、解构绑定、没有 ++
- §2.6 表达式与语句:if/match/loop 表达式(三目替代)
- §2.7 特色运算符:? 错误传播、范围 ..、求值顺序
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§2.1 运算符总览:与 C 的差异清单

Rust 的运算符绝大部分与 C 同名同义:算术 `+` `-` `*` `/` `%`、位 `&` `|` `^` `!` `<<` `>>`、比较 `==` `!=` `<` `>` `<=` `>=`、逻辑 `&&` `||`、赋值与复合赋值 `=` `+=` `--` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=`。差异集中在下面六点:

C 的特性	Rust 的处理
<code>++</code> / <code>--</code> 自增自减	删除!用 <code>x += 1</code> 替代(§2.5)
三目 <code>a ? b : c</code>	删除!用 if 表达式替代(§2.6)
逗号运算符 <code>a, b</code>	删除!Rust 不需要它
有符号溢出 <code>= UB</code>	确定性:debug panic、release 回绕(§2.2)
求值顺序未指定	明确规定:从左到右(§2.7)
无 <code>?</code> 运算符	新增 <code>?</code> :错误传播(§2.7)
无范围运算符	新增 <code>..</code> 与 <code>..=</code> (§2.7)
无解构赋值	新增 <code>let (a, b) = pair</code> 模式绑定(§2.5)

2.1.1 完整优先级表(从高到低)

优先级	运算符	与 C 的差异
最高	路径 ::、方法调用 .、下标 []、调用 ()	多出 :: 路径运算符
第2	?	Rust 新增(错误传播)
第3	一元:- ! * & &mut	多出借用运算符 &(取引用)
第4	as(类型转换)	C 用强制转换语法
第5	* / %	相同
第6	+ -	相同
第7	< < > >	相同
第8	&	相同
第9	^	相同
第10		相同
第11	== != < > <= >=	相同
第12	&&	相同
第13		相同
第14	=	Rust 新增(范围)
最低	= += -= 等赋值;return break	赋值是语句不是表达式

★ 重点: Rust 的 & 比 == 优先级高, C 则相反

C 的经典坑: `x & 1 == 0` 在 C 中解析为 `x & (1 == 0)`, 恒为 0, 必须写 `(x & 1) == 0`。Rust 的优先级表中 `&` 排在比较运算之上, `x & 1 == 0` 直接按 `(x & 1) == 0` 解析——教科书级的坑被优先级规则修掉了。但请记住一条工程铁律: 位运算与逻辑运算混用时, 永远加括号, 可读性优先于"我知道规则"。

§2.2 算术运算: 溢出与除法取模

2.2.1 溢出: C 的 UB 变成确定性行为

C 的有符号整数溢出是未定义行为: 编译器可能回绕、可能优化掉你的 if、可能产生任何结果。Rust 的规定清晰且可预期: debug 模式(默认 `cargo run`)下溢出直接 panic 并退出; release 模式(`cargo build --release`)下按二进制补码回绕。更关键的是, Rust 提供三种"显式溢出"方法, 把语义写进代码:

```
fn main() {
    // debug 模式:下面这行会 panic(先注释掉)
    // let x: i32 = i32::MAX + 1;

    // 需求是"回绕"(像 C 的 int):用 wrapping
    let a = i32::MAX.wrapping_add(1); // -2147483648
    // 需求是"到边界就停":用 saturating(饱和)
    let b = 255u8.saturating_add(10); // 255,不溢出
    // 需求是"溢出时报告":用 checked(返回 Option)
    let c = 100i32.checked_mul(2); // Some(200)
    let d = 100i32.checked_mul(1_000_000); // None:超过 i32 范围
    println!("{a} {b} {c:?} {d:?}");
}
```

场景	C	Rust
有符号溢出	UB(编译器的自由发挥)	debug panic / release 回绕
无符号回绕	定义良好(模 2^n)	相同,但 debug 下同样 panic
显式回绕	无标准写法	wrapping_add/sub/mul/...
饱和运算	手动 if 判断	saturating_add/sub/mul/...
溢出检查	靠 if (x > INT_MAX - y) 手写	checked_add 等返回 Option

2.2.2 除法与取模:与 C 相同的截断语义

整数除法向零截断、取模结果与被除数同号——**Rust 与 C99 完全一致** (Python 是向下取整,注意别混用)。另外两个 C 程序员注意不到的细节:**除零**在 Rust 中是 panic(debug 与 release 都 panic),不是 C 那种"得到 NaN 或崩溃或 UB";**取模除数也必须非零**。浮点除零得到无穷(与 C 相同,符合 IEEE 754):

```
fn main() {
    // 除法向零截断:与 C 相同
    println!("{}", 7 / 2); // 3
    println!("{}", -7 / 2); // -3(向零,不是向下:-3.5 的向零是 -3)
    // 取模:符号跟随被除数,与 C 相同、与 Python 不同
    println!("{}", 7 % 2); // 1
    println!("{}", -7 % 2); // -1(Python 会给出 1,注意!)
    println!("{}", 7 % -2); // 1

    // 浮点除零:IEEE 754 的无穷/NaN(与 C 相同)
    println!("{}", 1.0 / 0.0); // inf
    println!("{}", -1.0 / 0.0); // -inf
    println!("{}", 0.0 / 0.0); // NaN

    // 整数除零:panic(先注释掉再试)
    // let z = 10 / 0;
    // 生产代码用 checked_div 防炸
    let safe = 10i32.checked_div(0); // None
    println!("{safe:?}");
}
```

⚠ 易错点 1:把"溢出会悄悄回绕"的 C 习惯带进 Rust

在 C 里, `int a = INT_MAX; a++;` 静默变成 `INT_MIN`,你的程序继续跑,然后在某个诡异的地方爆炸。Rust 默认在 debug 模式下让这种溢出当场 panic,把错误暴露在它发生的位置——这是特性不是缺陷。如果线上需要回绕语义(哈希、计数器),请显式写 `wrapping_add`,并加注释说明为什么允许回绕;审计代码时一眼就能看清你的意图。

§2.3 位运算

2.3.1 全套位运算符

`& | ^ ! << >>` 与 C 语义一致。差异有二:① 位运算符只能用于整数类型 (C 里可以对指针/浮点做位操作,危险但合法;Rust 直接编译错误);② 右移区分类型:无符号数逻辑右移(高位补 0),有符号数算术右移(高位补符号位)——与 C 的"实现定义"不同,Rust 把它固定下来:

```
fn main() {
    let a: u8 = 0b1100_0011;
    let b: u8 = 0b1010_1010;
    println!("{:08b}", a & b);    // 10000010 按位与
    println!("{:08b}", a | b);    // 11101011 按位或
    println!("{:08b}", a ^ b);    // 01101001 按位异或
    println!("{:08b}", !a);       // 00111100 按位取反
    println!("{:08b}", a << 2);    // 00001100 左移:低位补 0,高位丢弃
    println!("{:08b}", a >> 2);    // 00110000 逻辑右移:高位补 0

    // 有符号右移是算术右移:负数补 1
    let n: i8 = -8;                // 二进制:11111000
    println!("{}", n >> 1);        // -4(补符号位)
    let u: u8 = 0b1000_0000;
    println!("{}", u >> 1);        // 64(补 0)

    // 注意:! 是一元位取反;逻辑非用 !b 吗?不,逻辑非是 ! 作用于 bool
    let flag = true;
    println!("{}", !flag);         // false(! 在 bool 上就是逻辑非)
}
```

2.3.2 生产场景:标志位与位掩码

```
// 权限标志:C 里典型写法是 #define READ (1<<0)
const READ: u32 = 1 << 0;    // 0b001
const WRITE: u32 = 1 << 1;   // 0b010
const EXEC: u32 = 1 << 2;    // 0b100

fn main() {
    let mut perms: u32 = 0;
    perms |= READ | WRITE;     // 置位:加上读和写
    perms &= !EXEC;            // 清位:去掉执行
    perms ^= WRITE;            // 翻转:写权限取反
    println!("0b{:03b}", perms); // 0b001:只剩 READ

    if perms & READ != 0 {     // 判断某一位
        println!("有读权限");
    }
    if perms & WRITE != 0 {
        println!("有写权限");    // 不会打印:WRITE 被翻转掉了
    }
}
```

工程建议:位运算的四个生产原则

- 标志位用常量命名,禁止裸数字: `READ = 1 << 0` 而非 `1`;C 的 `#define` 在 Rust 里对应 `const`
- 移位量必须是"无符号且小于位宽"的常量或已知值; `1u32 << 32` 在 Rust 中直接 panic,不会像 C 那样静默出错
- 位运算与比较混用一律加括号,哪怕你知道优先级
- 频繁用到位操作的专业领域(协议解析、编解码),用位字段结构体替代裸掩码,可读性翻倍(第14章 newtype 模式)

§2.4 比较与逻辑

2.4.1 == 的值语义与浮点比较

Rust 的 `==` 只做值比较:整数比数值、字符串比内容(`String` / `&str` 直接可比,不需要 C 的 `strcmp`)、结构体默认不可比——需要 `#[derive(PartialEq)]` (第6章)。C 的 "`==` 比较指针地址"与"比较字符串要 `strcmp`"的双轨制在 Rust 中不存在。浮点的坑与 C 相同:NaN 不等于任何数、`0.1 + 0.2 != 0.3`:

```
fn main() {
    // 字符串直接 ==:内容比较(不是指针比较)
    let a = String::from("hello");
    let b = String::from("hello");
    println!("{}", a == b);           // true

    // NaN 的自我否定:与 C 相同,但 Rust 可以显式判断
    let nan = f64::NAN;
    println!("{}", nan == nan);       // false
    println!("{}", nan.is_nan());     // true:官方推荐写法

    // 二进制浮点误差:0.1 + 0.2 是 0.30000000000000004
    let x = 0.1 + 0.2;
    println!("{}", x == 0.3);         // false

    // 正确姿势:比较差值的绝对值是否小于阈值
    let eps = 1e-9;
    println!("{}", (x - 0.3).abs() < eps); // true
}
```

2.4.2 短路与无隐式真值

`&&` 与 `||` 的短路求值语义与 C 完全一致(左边就能决定结果时,右边不求值)。但 **条件必须是 bool**:C 的 `if (ptr)`、`if (n)`、`while (c)` 在 Rust 全部编译错误,必须显式写出判断。另外 `&&` / `||` 返回 `bool`,绝不返回操作数(与 Python/JS 不同):

```
fn main() {
    let a = false;
    if a && side_effect() {           // side_effect 不会被调用
        println!("不会执行");
    }
    let b = true;
    if b || side_effect() {          // 同上:短路
        println!("会执行");
    }

    // C 的 if (ptr) / if (fd) 必须显式化:
    let fd: Option<i32> = Some(3);    // 第1章的 Option:要么有值要么没有
    if fd.is_some() {
        println!("文件描述符存在");
    }

    // 逻辑运算结果永远是 bool,不能当数用
    let r = 1 < 2 && 3 < 4;
    println!("{}", r);               // true
}

fn side_effect() -> bool {
    println!("side_effect 被调用了!");
    true
}
```

⚠ 易错点 2: 短路没生效的错觉与链式比较

两件事容易搞混: ① `&&` 的右操作数必须是表达式而不是语句, `a && b = 1` 这种 C 写法在 Rust 是编译错误; ② C 的 `a < b < c` (先算 `a < b` 得 0/1, 再与 `c` 比) 在 Rust 中同样编译错误——必须写成 `a < b && b < c`。这两条都是“编译器强制你写清楚”, 不是陷阱而是保护。

§2.5 赋值: 复合赋值、解构绑定、没有 ++

2.5.1 赋值是语句, 不是表达式

C 的经典惯用法 `while ((c = getchar()) != EOF)` 依赖“赋值是表达式”。Rust 中**赋值不产生值**, 它是语句——上面这行直接编译错误。连锁反应: 连锁赋值 `a = b = c`、条件里赋值、`if (x = f())` 全部消失。好处是杜绝了“想写 `==` 却写成 `=`”的经典事故——编译器会拒绝这种代码。

2.5.2 复合赋值: 与解构绑定

复合赋值 `+= -= *= /= %= &= |= ^= <<= >>=` 与 C 语义相同, 且**左侧的位置表达式只求值一次** (C 里 `a[i++] += 1` 是 UB, 因为它两次计算下标)。Rust 还允许在 `let` 中解构绑定:

```
fn main() {
    let mut x = 10;
    x += 5;           // 15
    x *= 2;           // 30
    x >>= 1;          // 15(算术右移)
    x ^= 0xFF;        // 240
    println!("{}", x);

    // 解构绑定: 一次把元组拆开
    let pair = (3, 7);
    let (a, b) = pair; // a = 3, b = 7
    println!("{a} {b}");

    // 解构数组与带 _ 的占位
    let [first, second, _] = [1, 2, 3];
    println!("{first} {second}");
}
```

2.5.3 没有 ++ 与 --: 为什么? 替代是什么?

Rust 刻意不提供 `++ / --`。理由有二: ① 前置/后置的返回值差异 (`x++` 返回旧值) 是 C 里 `i = i++ + i++` 一类未定义行为与可读性灾难的源头; ② Rust 推崇“表达式只做一件事”, `x += 1` 意图比 `x++` 更直白。替代写法: 普通自增 `x += 1`; for 循环用范围 `for i in 0..n` (第3章); 先取旧值再自增这种需求, 用显式两行:


```
fn main() {
    let mut count = 0;
    count += 1;           // 替代 count++
    count -= 1;           // 替代 count--

    // 需要"旧值"的场景(C 的 y = x++):
    let mut x = 5;
    let y = x;             // 先取旧值
    x += 1;                // 再自增
    println!("{y} {x}");   // 5 6

    // C 的 for (i = 0; i < n; i++):
    for i in 0..5 {        // 第3章详解,这里先感受
        print!("{i} ");
    }
    println!();
}
```

⚠ 易错点 3:把 C 的"语句即表达式"习惯搬过来

三类代码在 Rust 中是编译错误,而且都曾经是 C 的"日常":① `while ((c = getchar()) != EOF)` ——赋值不是表达式;② `x = y = z` 连锁赋值;③ `return (x = 5)` 这种"赋值兼返回"。改写思路都一样:把赋值拆成语句,把值存进变量再用。难受两天,换来的是"条件里绝不混赋值"的终身安全。

§2.6 表达式与语句:if/loop/match 都是表达式

2.6.1 C 的三目?: 在 Rust 中是 if 表达式

Rust 没有 `?:`,但 `if` 本身就是表达式:每个分支是一个代码块,块的最后一个表达式(不带分号)就是分支的值。要求:所有分支类型必须一致(否则编译错误)。注意 `else` 不可省略——if 表达式必须给出所有情况的值:

```
fn main() {
    let score = 75;
    // C: int grade = score >= 60 ? 1 : 0;
    let grade = if score >= 60 { "及格" } else { "不及格" };
    println!("{grade}");

    // if-else 链:C 的 else if 链同样成立
    let level = if score >= 90 {
        "A"
    } else if score >= 80 {
        "B"
    } else if score >= 70 {
        "C"
    } else {
        "D"
    };
    println!("{level}");

    // 只用 if 做"副作用"时,分支返回 () 即可(后面不带值)
    let mut v = Vec::new();
    if score > 90 { v.push("优秀"); }
    println!("{:?}", v);
}
```

2.6.2 代码块也是表达式:{} 有值

```
fn main() {
    // 代码块的值 = 最后一个表达式(不带分号)
    let x = {
        let y = 3;
        let z = 4;
        y * z          // 没有分号:这就是代码块的值
    };
    println!("{x}");    // 12

    // 带分号则是 (),常见于"初始化一段临时逻辑"
    let _nothing = { let t = 1; t + 1; }; // 值为 ()
    println!("hello");
}
```

★ 重点:分号决定"是表达式还是语句"

这是全书写 Rust 最常见的"卡亮点": 块内最后一个表达式带分号 → 块的值是 ();不带分号 → 块的值是表达式的值。编译错误 "expected `i32`, found `()`" 十有八九就是这个:你在 `if` 分支末尾多写了分号。遇到立刻检查:最后一个表达式有没有多余的分号。

§2.7 特色运算符:?. 范围、求值顺序

2.7.1 ? 运算符:错误传播

? 是 Rust 的标志性运算符:当表达式返回 Result 时,成功则取出值继续,失败则立即把错误"向上返回"。它消灭的正是 C 代码里铺天盖地的 `if (err) return err;` 样板(第9章详解,这里先看形态):

```
use std::num::ParseIntError;

// ? 要求所在函数返回 Result(或 Option)
fn parse_and_double(text: &str) -> Result<i32, ParseIntError> {
    let n: i32 = text.trim().parse()?; // 失败:立刻 return Err
    Ok(n * 2)                          // 成功:拿到 n 继续算
}

fn main() {
    // 调用方可以继续用 ? 向上传,也可以 match 处理
    match parse_and_double(" 21 ") {
        Ok(v) => println!("结果:{v}"), // 42
        Err(e) => println!("错误:{e}"),
    }
    match parse_and_double("abc") {
        Ok(v) => println!("结果:{v}"),
        Err(_) => println!("解析失败"),
    }
}
```

2.7.2 范围运算符 .. 与 ..=

..`表示半开区间(含头不含尾), ..= 表示闭区间。它是循环与切片(第3章)的基石,C 的 for (i = 0; i < n; i++) 在 Rust 中写作 for i in 0..n:`

```
fn main() {
    for i in 1..5 { print!("{i} "); } // 1 2 3 4
    println!();
    for i in 1..=5 { print!("{i} "); } // 1 2 3 4 5
    println!();

    // 与数组切片结合(第1章见过)
    let arr = [10, 20, 30, 40, 50];
    println!("{:?}", &arr[1..3]); // [20, 30]
    println!("{:?}", &arr[..2]); // [10, 20]
    println!("{:?}", &arr[2..]); // [30, 40, 50]
}
```

2.7.3 求值顺序:C 的"未指定"变成"从左到右"

C 里 `f() + g()` 先调谁、`a[i] = i++` 先算谁,都是未指定行为;Rust 明确规定:二元运算符的操作数、函数实参都按从左到右求值;赋值先求值右侧,再求值左侧。副作用顺序可预测,这是编写"表达式内带副作用"代码(虽然不建议)时安心的前提:

```
fn main() {
    // 从左到右:先打印 "左" 再打印 "右"
    let r = left() + right();
    println!("结果:{r}");
}

fn left() -> i32 {
    println!("左操作数先求值");
    1
}

fn right() -> i32 {
    println!("右操作数后求值");
    2
}
```

🚀 工程建议:表达式的生产准则

- 一行只干一件事:避免"表达式里带副作用"(如 print 里调用函数、下标里 i++)——Rust 虽规定顺序,但可读性仍差
- 优先用 let 中间变量拆解长表达式,比注释更有效
- 算术溢出有明确语义后,审计代码时重点找 as 截断与 unwrap(第9章),而不是猜测溢出行为
- 用 clippy(cargo clippy)做静态检查:它会对 `x & 1 == 0` 这类写法给出更可读的替代建议

本章速查表

主题	结论 / 写法
算术	+ - * / % 与 C 相同;除法向零截断;-7 % 2 = -1(与 Python 不同)
溢出	debug 模式 panic;release 回绕;wrapping/saturating/checked 三种显式选择
除零	整数除零、取模除零都是 panic;checked_div 返回 Option;浮点除零得 inf/NaN
自增自减	无 ++/--;x += 1;需要旧值就"先取后加"两行
位运算	& ^ ! << >>;只用于整数;无符号逻辑右移、有符号算术右移
位标志	const 命名 + 置位 / &= ! 清位 / ^ 翻转 / & != 0 判断
比较	== 是值比较:String 直接 ==;NaN != NaN;浮点比较用 (a-b).abs() < eps
逻辑	&& 短路且返回 bool;条件必须是 bool,无隐式真值
赋值	赋值是语句不是表达式;复合赋值左侧只求值一次
解构	let (a, b) = pair;let [a, b, _] = arr;
三目	无 ?::用 if 表达式:let m = if a { x } else { y };
表达式	if/match/loop/{ } 都是表达式;最后无分号者即值;分支类型必须一致
特色	? 错误传播(函数须返回 Result/Option);.. 半开区间、..= 闭区间
优先级	& 高于 == (C 相反);位运算与比较混用一律加括号
求值顺序	从左到右;赋值先右后左;无 C 的"未指定"

最小必会清单

- 能说出 Rust 从 C 删掉的四个运算符/语法(++ -- ?: 逗号、赋值当表达式)与新增的两个(.. 与 ?)
- 能写出 debug/release 两种模式下的溢出行为,以及 wrapping/saturating/checked 三种方法名
- 能口算 $-7 / 2$ 与 $-7 \% 2$,并解释与 Python 的差异
- 能把 C 的 `grade = score >= 60 ? 'A' : 'F'` 改写为 Rust if 表达式
- 能解释 `x & 1 == 0` 在 C 与 Rust 中的解析差异
- 能写出浮点比较的正确姿势并解释为什么
- 能解释为什么 `while ((c = getchar()) != EOF)` 在 Rust 中无法编译
- 能给出一段用位运算做权限标志的完整代码
- 能说出 if 表达式所有分支的类型要求,以及"分号决定表达式还是语句"
- 能说出 Rust 规定的求值顺序,以及它与 C 的差异

自测题

Q 1. `let x = i32::MAX; let y = x + 1;` 在 debug 模式与 release 模式下分别发生什么?C 里 `int y = INT_MAX + 1;` 呢?

✓ 参考答案

Rust debug:panic(attempt to add with overflow),进程终止;release:回绕为 `i32::MIN`。C 是未定义行为:结果不可预测,取决于编译器与优化。Rust 用"确定性"取代"UB":要么在出错现场终止,要么按补码回绕,且可用 `wrapping_add` 显式声明意图。

Q 2. 计算 $-7 / 2$ 与 $-7 \% 2$,并说明与 Python 的差异。

✓ 参考答案

$-7/2 = -3$ (向零截断), $-7\%2 = -1$ (符号跟随被除数),与 C99 相同。Python 里 $-7//2 = -4$ (向下取整), $-7\%2 = 1$ 。差异根源:Python 要求" $a = (a//b)*b + a\%b$ "且余数非负;Rust/C 允许余数为负。写跨语言移植代码时务必显式测试。

Q 3. 用 Rust 表达 C 的 `int m = a > b ? a : b;`,并要求 if 表达式的两个分支类型一致。若写成 `let m = if a > b { a } else { "大" };` 会怎样?

✓ 参考答案

`let m = if a > b { a } else { b };`。分支类型不一致(`i32` vs `&str`)会编译错误,报 "if and else have incompatible types"。这正是 Rust 强制所有分支必须给出同类型值的体现。

Q 4. `x & 1 == 0` 在 C 与 Rust 中的解析分别是什么?为什么会这样?

✓ 参考答案

C 中 `==` 优先级高于 `&`, 解析为 `x & (1 == 0)`, 即 `x & 0`, 恒为 0——经典 bug 来源, 必须写 `(x & 1) == 0`。Rust 中 `&` 优先级高于 `==`, 解析为 `(x & 1) == 0`, 语义正确。工程上仍建议加括号。

Q 5. 为什么 Rust 没有 ++ 与 --? C 的 `y = x++` 如何改写?

✓ 参考答案

前置/后置返回值差异带来大量可读性灾难与未定义行为(如 `i = i++ + i++`)。Rust 用 `x += 1` 表达自增; 需要旧值时显式两行: `let y = x; x += 1;`。牺牲一点简洁, 换取“表达式不再隐藏副作用”。

Q 6. 下面代码输出什么? `let mut x = 4; x >>= 1; x |= 1; println!("{}", x);` 有符号与无符号右移有什么区别?

✓ 参考答案

`x = 2` (4 逻辑右移 1 位), 再 `2 | 1 = 3`, 输出 3。有符号右移是算术右移(负数补 1), 无符号右移是逻辑右移(补 0): `-8i8 >> 1 = -4`, 而 `0b1000_0000u8 >> 1 = 64`。C 中这是实现定义行为, Rust 固定了规则。

章末实验题: 表达式求值计算器

实验 2: 表达式求值计算器

实验目标: 实现一个命令行表达式计算器, 输入字符串如 `"2 + 3 * 4"`、`"1 << 8"`、`"-5 % 3"`, 输出计算结果; 正确处理优先级、括号、错误报告与溢出。

实验要求:

- 任务 1: 分词器 `tokenize`: 把输入拆成数字与运算符 Token (覆盖: 字符遍历、整数字面量解析、Option/Result 雏形)
- 任务 2: 递归下降解析器, 按优先级分层 (覆盖: 优先级表 §2.1、一元负号、括号)
- 任务 3: 算术运算用 `checked_add/checked_sub/checked_mul` 防溢出, 除零与取模除零报错 (覆盖: 溢出三兄弟 §2.2)
- 任务 4: 位运算符 `& | ^ << >>` 接入解析器 (覆盖: 位运算 §2.3、移位量校验)
- 任务 5: `main` 中对输入做短路校验并逐条执行测试用例 (覆盖: 短路 `&& ||` §2.4、复合赋值 `+=` 等、`match` 处理 Result)
- 任务 6: 用 `if` 表达式与 `==` 对比两个表达式的计算结果并输出 (覆盖: `if` 表达式与比较 §2.5–2.6)

实验环境: `cargo new calc_rs`, 代码写入 `src/main.rs`, `cargo run` 运行; 自备 10 条测试表达式 (含故意写错的)。

第一步 一步: 先写 Token 枚举与 `tokenize`; 注意 `<<` 是两个字符, 读到单个 `<` 要报错

第二步 二步: Parser 持有一个 Token 数组与下标; 每个 `parse_xxx` 函数返回 `Result<i64, String>`, 内部用 ? 传播下层错误

第三步 三步: 按 `|` 最低, 然后 `^`、`&`、移位、加减、乘除、一元负号最高"的顺序嵌套实现

第四步 四步: `main` 里逐个跑测试用例, 用 `match` 打印 "表达式 = 结果" 或错误信息

第五步 五步: 追加任务 6: 解析两个表达式并用 `if` 表达式输出比较结果

✓ 参考答案要点

```
// ===== 实验 2:表达式求值计算器 =====
// 覆盖知识点:优先级与结合性($2.1)、溢出处理($2.2)、除法取模($2.2)、
//          位运算($2.3)、短路与复合赋值($2.4/2.5)、
//          if 表达式与 == 比较($2.6)、? 错误传播与范围($2.7)

#[derive(Debug, Clone, PartialEq)]
enum Token {
    Num(i64),
    Plus, Minus, Star, Slash, Percent,
    Amp, Pipe, Caret, Shl, Shr,
    LParen, RParen,
}

/// 分词:把源字符串切成 Token 序列
fn tokenize(src: &str) -> Result<Vec<Token>, String> {
    let chars: Vec<char> = src.chars().collect();
    let mut tokens = Vec::new();
    let mut i = 0;
    while i < chars.len() {
        let c = chars[i];
        match c {
            ' ' | '\t' => {} // 空白跳过
            '0'..'9' => {
                let mut n: i64 = 0;
                while i < chars.len() && chars[i].is_ascii_digit() {
                    n = n.checked_mul(10).ok_or("数字字面量溢出")?
                        .checked_add(chars[i] as i64 - '0' as i64)
                        .ok_or("数字字面量溢出")?;
                    i += 1;
                }
                tokens.push(Token::Num(n));
                continue; // 已前进,跳过底部 i += 1
            }
            '+' => tokens.push(Token::Plus),
            '-' => tokens.push(Token::Minus),
            '*' => tokens.push(Token::Star),
            '/' => tokens.push(Token::Slash),
            '%' => tokens.push(Token::Percent),
            '&' => tokens.push(Token::Amp),
            '|' => tokens.push(Token::Pipe),
            '^' => tokens.push(Token::Caret),
            '<' => {
                if chars.get(i + 1) == Some(&'<') { tokens.push(Token::Shl); i += 1; }
                else { return Err(format!("位置 {i}:单独的 < 不是合法运算符")); }
            }
            '>' => {
                if chars.get(i + 1) == Some(&'>') { tokens.push(Token::Shr); i += 1; }
                else { return Err(format!("位置 {i}:单独的 > 不是合法运算符")); }
            }
            '(' => tokens.push(Token::LParen),
            ')' => tokens.push(Token::RParen),
            _ => return Err(format!("不支持的字符 '{c}'")),
        }
        i += 1;
    }
}
```

```

    Ok(tokens)
}

/// 递归下降解析器:优先级从低到高 | → ^ → & → 移位 → 加减 → 乘除 → 一元负号
struct Parser {
    tokens: Vec<Token>,
    pos: usize,
}

impl Parser {
    fn new(tokens: Vec<Token>) -> Self {
        Parser { tokens, pos: 0 }
    }

    fn peek(&self) -> Option<&Token> { self.tokens.get(self.pos) }

    fn next(&mut self) -> Option<Token> {
        let t = self.tokens.get(self.pos).cloned();
        self.pos += 1;
        t
    }

    fn parse_or(&mut self) -> Result<i64, String> {
        let mut left = self.parse_xor()?;
        while self.peek() == Some(&Token::Pipe) {
            self.next();
            let right = self.parse_xor()?;
            left |= right;           // 复合赋值
        }
        Ok(left)
    }

    fn parse_xor(&mut self) -> Result<i64, String> {
        let mut left = self.parse_and()?;
        while self.peek() == Some(&Token::Caret) {
            self.next();
            let right = self.parse_and()?;
            left ^= right;
        }
        Ok(left)
    }

    fn parse_and(&mut self) -> Result<i64, String> {
        let mut left = self.parse_shift()?;
        while self.peek() == Some(&Token::Amp) {
            self.next();
            let right = self.parse_shift()?;
            left &= right;
        }
        Ok(left)
    }

    fn parse_shift(&mut self) -> Result<i64, String> {
        let mut left = self.parse_add()?;
        loop {

```



```

        match self.peek() {
            Some(Token::Shl) => {
                self.next();
                let r = self.parse_add()?;
                let r = u32::try_from(r).map_err(|_| "移位置必须非负")?;
                left = left.checked_shl(r).ok_or("左移溢出")?;
            }
            Some(Token::Shr) => {
                self.next();
                let r = self.parse_add()?;
                let r = u32::try_from(r).map_err(|_| "移位置必须非负")?;
                left = left.checked_shr(r).ok_or("移位置过大")?;
            }
            _ => break,
        }
    }
    Ok(left)
}

fn parse_add(&mut self) -> Result<i64, String> {
    let mut left = self.parse_mul()?;
    loop {
        match self.peek() {
            Some(Token::Plus) => {
                self.next();
                let r = self.parse_mul()?;
                left = left.checked_add(r).ok_or("加法溢出")?;
            }
            Some(Token::Minus) => {
                self.next();
                let r = self.parse_mul()?;
                left = left.checked_sub(r).ok_or("减法溢出")?;
            }
            _ => break,
        }
    }
    Ok(left)
}

fn parse_mul(&mut self) -> Result<i64, String> {
    let mut left = self.parse_unary()?;
    loop {
        match self.peek() {
            Some(Token::Star) => {
                self.next();
                let r = self.parse_unary()?;
                left = left.checked_mul(r).ok_or("乘法溢出")?;
            }
            Some(Token::Slash) => {
                self.next();
                let r = self.parse_unary()?;
                if r == 0 { return Err("除数为零".into()); }
                left /= r; // 截断除法,与 C 相同
            }
            Some(Token::Percent) => {

```

```

        self.next();
        let r = self.parse_unary()?;
        if r == 0 { return Err("取模除数为零".into()); }
        left %= r;
    }
    _ => break,
}
}
Ok(left)
}

fn parse_unary(&mut self) -> Result<i64, String> {
    if self.peek() == Some(&Token::Minus) {
        self.next();
        let v = self.parse_unary()?;
        return v.checked_neg().ok_or("取负溢出");
    }
    match self.next() {
        Some(Token::Num(n)) => Ok(n),
        Some(Token::LParen) => {
            let v = self.parse_or()?; // 括号内重新按优先级解析
            match self.next() {
                Some(Token::RParen) => Ok(v),
                _ => Err("缺少右括号".into()),
            }
        }
        _ => Err("语法错误:缺少操作数".into()),
    }
}

}

/// 顶层入口:解析整条表达式,并拒绝"多余内容"
fn eval(src: &str) -> Result<i64, String> {
    let tokens = tokenize(src)?; // ? 错误传播
    let mut p = Parser::new(tokens);
    let value = p.parse_or()?;
    if p.peek().is_some() {
        return Err("语法错误:表达式后有多余内容".into());
    }
    Ok(value)
}

fn main() {
    // 短路校验:空输入或超长输入直接跳过(覆盖 && 短路)
    let cases = [
        "2 + 3 * 4",
        "(2 + 3) * 4",
        "10 / 3",
        "-7 / 2",
        "-7 % 2",
        "255 & 240",
        "1 | 2 | 4",
        "1 << 8",
        "8 >> 1",
        "-5 + 3",
    ];
}

```

```

    "100 / 0",
    "10 % 0",
    "2 +",
    "1 + 2 +",
    "",
];
for case in cases {
    let t = case.trim();
    if t.is_empty() || t.chars().count() > 64 { // 短路:左边为真就不再算右边
        println!("跳过空/超长输入");
        continue;
    }
    match eval(t) {
        Ok(v) => println!("{case:<10} = {v}"),
        Err(e) => println!("{case:<10} 错误: {e}"),
    }
}

// 任务 6:比较两个表达式的值,用 if 表达式输出结论(覆盖 == 与 if 表达式)
let a = eval("1 + 2 * 3").unwrap_or(0);
let b = eval("(1 + 2) * 3").unwrap_or(0);
let conclusion = if a == b { "两个表达式结果相同" } else { "结果不同" };
println!("{a} 与 {b}:{conclusion}");
}

```

设计说明:整个解析器刻意**不含任何 if/else 嵌套超过一层**——优先级靠函数嵌套表达,错误靠 ? 向上传播,这是 Rust 表达式文化的浓缩:每个 parse_xxx 返回 Result,内部零样板错误处理。运行后把 "255 & 240" 改成 "255 & 0xFF" 试试?你的 tokenize 目前不支持十六进制——这就是任务清单里"自备测试表达式"的意义:让程序自己暴露薄弱点,再回头修。

下一章预告:第3章 流程控制——match 的穷尽性检查、if let/while let 糖、带标签的 break、for 与迭代器。C 的 switch 只能等值比较整型,Rust 的 match 能把任意类型拆开看——那才是真正的"模式匹配"。